

Time, Clocks & Coordination

in Distributed Systems

Weeks 7–8 | Distributed Systems Fundamentals

[Instructor Name] · [Course Code] · [Date]

Learning Objectives

- 1 Define the happens-before relation and explain why physical time alone is insufficient for ordering events in distributed systems.
- 2 Implement and trace Lamport scalar clocks and vector clocks to determine causal relationships (including concurrency).
- 3 Compare centralized vs. distributed approaches to mutual exclusion and leader election and evaluate their trade-offs.
- 4 Explain the purpose and high-level operation of distributed snapshots (Chandy-Lamport).
- 5 Describe the role of coordination services such as ZooKeeper and how modern systems (e.g., Kafka) have evolved.

Agenda

Week 7 – Time & Clocks

- Physical time problems
- Lamport & Vector clocks
- Live examples & activity

Week 8 – Coordination

- Mutual exclusion & leader election
- Distributed snapshots
- Coordination services (ZooKeeper)

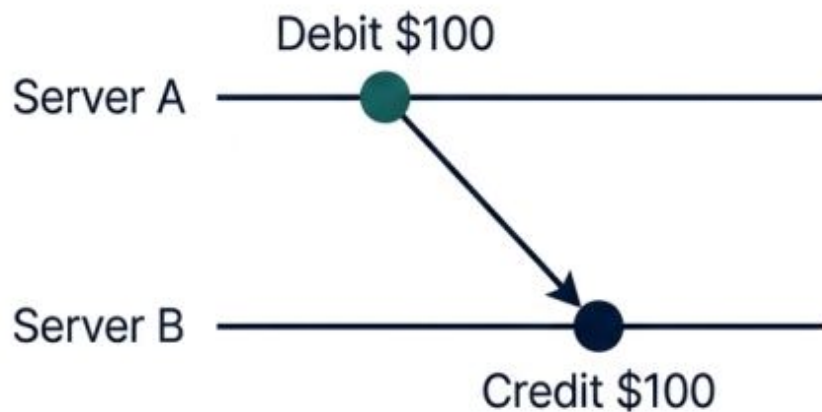
Hands-on Lab

Vector clock implementation in Python + comparison of clock synchronization algorithms

Why Time is Hard: The Banking Paradox

- Transfer \$100 from Account A → Account B.
- Server A debits \$100.
- Server B credits \$100.

The Problem: How do we prove the debit happened *before* the credit across the world?



Key Insight: There is no single global 'now'.

Physical Clocks & Drift

Why two machines never agree on time

- Quartz crystal oscillators drift (even atomic clocks have tiny error)
- Typical drift: 1–10 ms per day on commodity hardware
- Skew = difference between two clocks at one moment
- Drift = rate at which skew grows over time

Real Impact

A 1 ms drift per day becomes noticeable after weeks in a large cluster.
This breaks ordering guarantees needed for correctness.

The Illusion of Physical Synchronization

Quartz Drift: Commodity hardware clocks drift 1–10 ms daily.

NTP Limits: Network Time Protocol yields microsecond accuracy in data centers, but cannot guarantee total ordering.

Spanner's Patch: Google TrueTime adds uncertainty intervals $[T, T+\epsilon]$ and forces transactions to wait it out.

Even Google does not trust raw physical time.
Logical mechanisms are required for absolute correctness.

NTP – Synchronizing Physical Time

Network Time Protocol (RFC 5905)

- Hierarchical stratum model (Stratum 0 = atomic/GPS clocks)
- Clients exchange timestamps with servers
- Calculate offset and round-trip delay
- Typical accuracy: 1–10 ms on the internet; microseconds inside data centers

Limitation

NTP is “good enough” for many applications but cannot give strong total ordering guarantees across machines. We still need logical clocks on top.

When Physical Time Isn't Enough

Case Study: Google Spanner's TrueTime

- Combines GPS + atomic clocks
- Returns time with an uncertainty interval (e.g., $[T, T+\epsilon]$)
- Transactions wait out the uncertainty before committing
- Enables external consistency (stronger than snapshot isolation)

Key Takeaway

Even Google doesn't trust raw physical time. They add uncertainty and wait. This shows why logical mechanisms are still essential on top of physical clocks.

Logical Clocks – Lamport's Insight (1978)

The Happens-Before Relation ($\rightarrow$)

We don't need wall-clock time. We need causality.

Event $a \rightarrow b$ if:

- a and b are on the same process and a happened before b , OR
- a is the send of a message and b is the receive of that message

This relation captures causality without needing synchronized physical clocks.

Lamport Clock Algorithm + Walkthrough

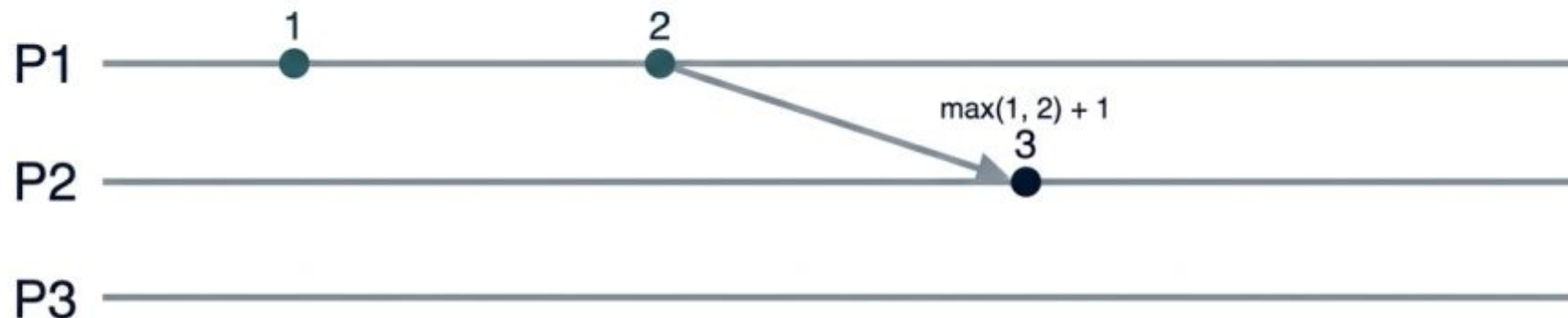
Simple Rules

1. Each process maintains a counter C
2. On local event: $C := C + 1$
3. On sending message: attach current C
4. On receiving message: $C := \max(C, \text{received_C}) + 1$

Algorithm: Lamport Clocks

Each process maintains a counter C .

1. On local event: $C := C + 1$
2. On sending message: Attach current C to message.
3. On receiving message: $C := \max(C, \text{received_C}) + 1$



Lamport Clock Algorithm + Walkthrough

Lamport clocks give a total order but...

Two events can have $C(a) < C(b)$ even when they are concurrent (no causal relationship).

Limitation: We cannot distinguish “a happened before b” from “a and b are concurrent.”

This is why we need vector clocks.

Vector clocks add precision by tracking knowledge per process and can detect true concurrency.

The Catch: Invisible Concurrency

Lamport clocks give a total order, but create false positives.

If $a \rightarrow b$, then $C(a) < C(b)$ (TRUE)

If $C(a) < C(b)$, it does NOT mean $a \rightarrow b$ (FALSE)

We cannot distinguish “Event A happened before Event B” from “Event A and Event B are concurrent.”

Vector Clocks – Adding Structure

One counter per process

Each process i maintains a vector VC of size N (one entry per process).

Intuition: “What I know about what everyone else has seen.”

Update rules:

- On local event: increment own component
- On receive: take component-wise max, then increment own

The vector captures distributed knowledge across the system.

Vector Clock Walkthrough

Step-by-step 3-node example

Show a concrete execution with send/receive/merge steps and final vectors.

Comparison rule (simple version):

- $VC(a) < VC(b)$ if every component of $a \leq$ corresponding component of b and at least one is strictly $<$
- Otherwise the events are concurrent

Recommended Visual

Side-by-side comparison of the same execution using Lamport vs Vector clocks. Show how vector clocks correctly identify concurrent events.

Algorithm: Vector Clocks

Each process i maintains a vector VC of size N .

1. On local event: Increment own component $VC[i] := VC[i] + 1$
2. On sending message: Attach current VC to message.
3. On receiving message:
 - Take component-wise max of local and received vectors.
 - Increment own component $VC[i] := VC[i] + 1$

Detecting Concurrency

Compare Vector A and Vector B:

A happened before B ($A < B$):

$$[1, 0, 0] < [1, 1, 0]$$

Every component of A is $\leq$ corresponding component of B,
AND at least one is strictly $<$.

B happened before A ($B < A$):

Every component of B is $\leq$ corresponding component of A,
AND at least one is strictly $<$.

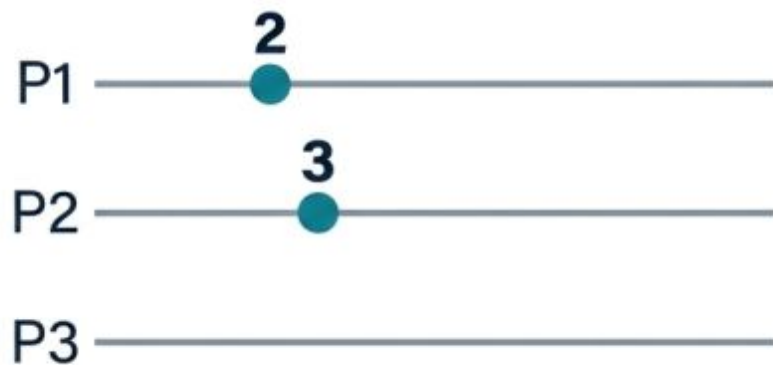
Concurrent ($A \parallel B$):

$$[1, 0, 0] \parallel [0, 1, 0]$$

Neither vector is strictly less than or equal to the other.

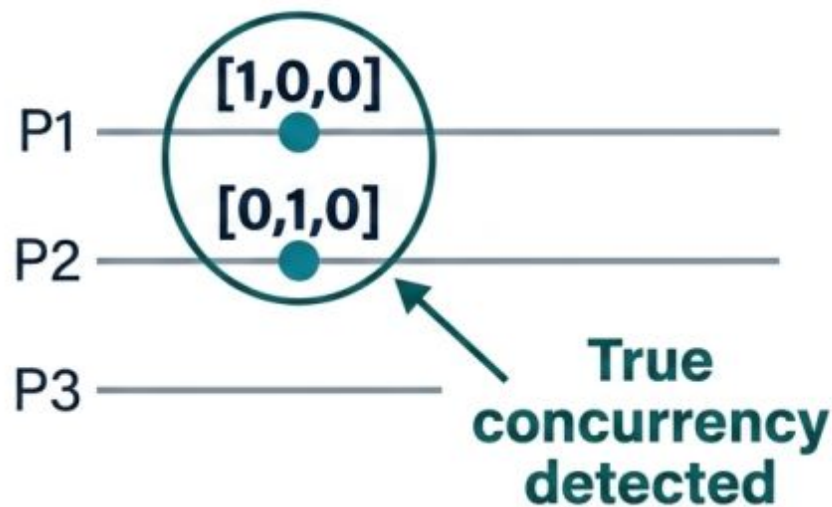
Visualizing the Difference

LAMPORT CLOCK TRACE



False Sequential Values

VECTOR CLOCK TRACE



Activity: Compare These Clocks

Mini in-class exercise (5 minutes)

You are given two vector clock snapshots from a 3-process system.

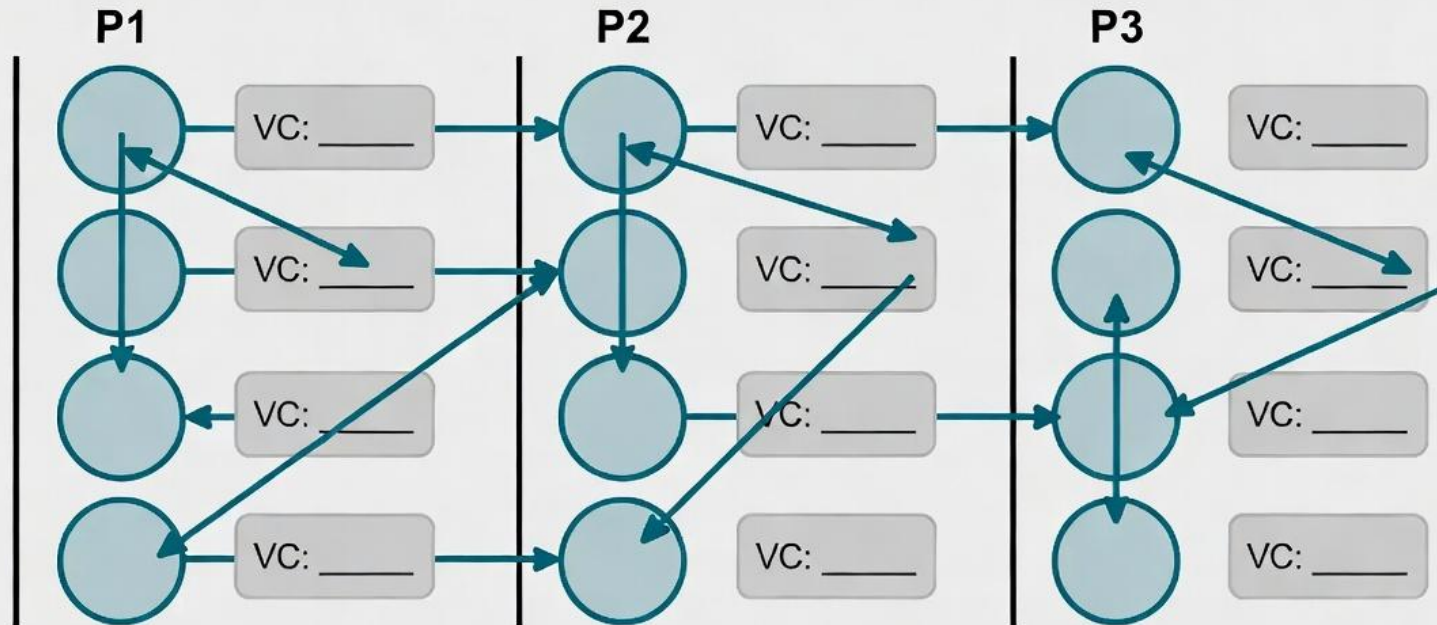
Task (pairs): Decide for each pair of events whether they are:

- Causally ordered (one happened before the other), OR
- Concurrent

Discussion Prompt

Why does being able to detect concurrency matter for a distributed database or collaborative editing system?

Activity: Trace & Compare - Vector Clocks (Student Version)



Evaluate these event pairs:

1. Causal ordering *find* between processes?
2. Concurrency between events on different process?

Evaluate these event pairs:

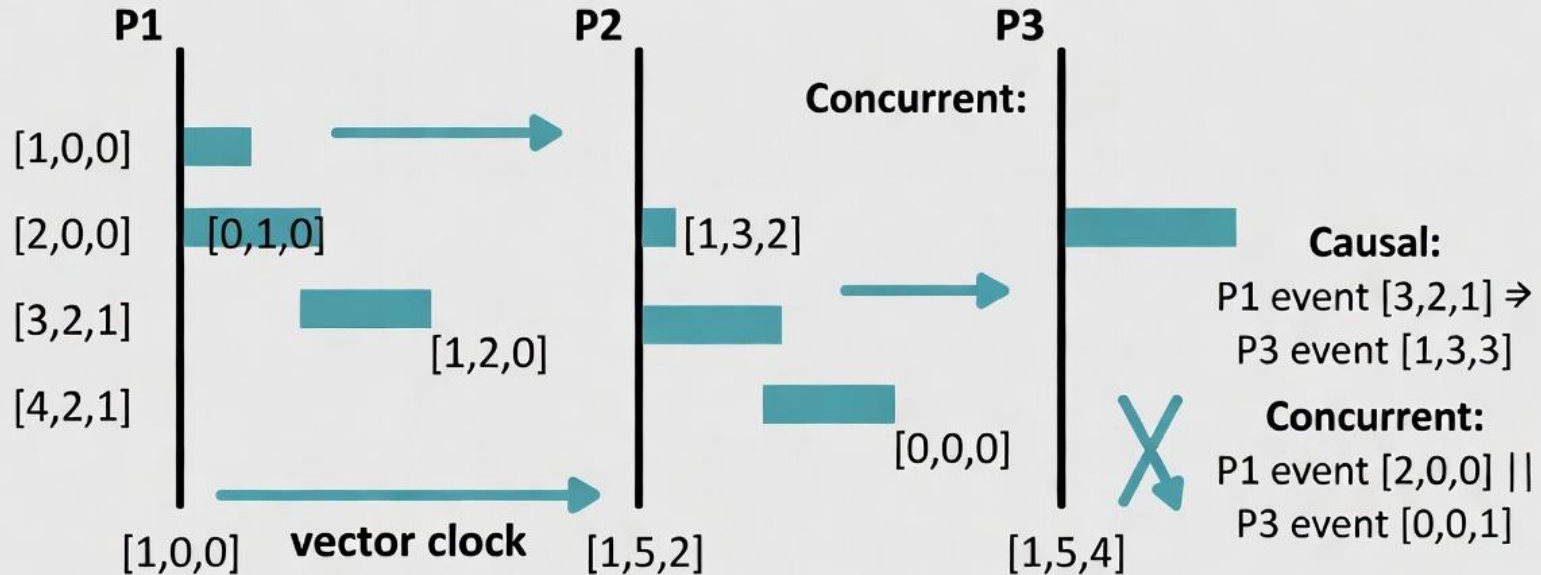
1. Causal ordering between on different processes?
2. Concurrency on different process?

Discussion Question: Why is detecting concurrency important for distributed databases and collaborative editing systems?

Activity: Trace & Compare - Vector Clocks

(Example Solution with Annotations)

Example Solution



Evaluate these event pairs:

1. Are Event on P2 and Event on P3 causally ordered or concurrent?
2. Are Event on P1 and Event on P3 causally ordered or concurrent?

Discussion Question: Why is detecting concurrency important for distributed databases and collaborative editing systems?

Distributed Mutual Exclusion – The Problem

Why shared-resource locking is hard with no shared memory

Multiple processes need exclusive access to a resource (file, database row, printer, etc.).

Challenges:

- No shared memory
- No global clock
- Messages can be lost or delayed
- Processes can crash

Relate back to Week 4 RPC: Even if you can call a remote lock(), how do you know it's safe?

The Illusion of Shared State



Resource locking is uniquely difficult without shared memory.

- No global wall-clock time
- Unpredictable message latency or total loss
- Inevitable process crashes

Even if a remote lock() succeeds (via RPC), proving it is globally safe requires external coordination.

Mutex Approaches (High-Level Comparison)

Approach	How it works	Pros	Cons
Centralized	One coordinator grants locks	Simple	Single point of failure
Token-ring	Token circulates; holder has lock	Fair, no starvation	Token loss is painful
Ricart-Agrawala	Timestamp-based voting	Fully distributed	$2(N-1)$ messages per request

Modern systems often use a coordination service (ZooKeeper, etcd, Consul) instead of implementing these from scratch.

Leader Election (High-Level)

Why systems need a single coordinator

- Someone must decide “who goes first”
- Common in replication, task assignment, configuration management

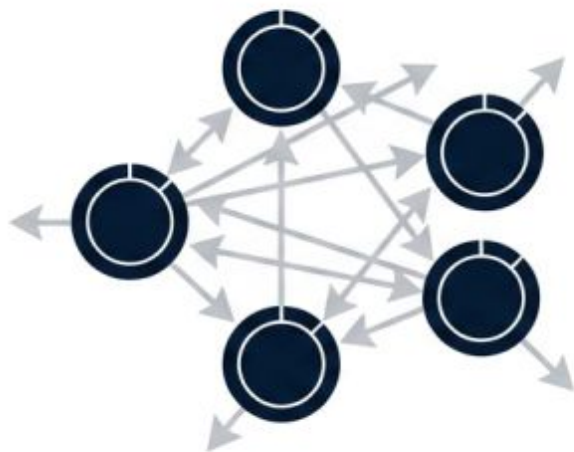
Classic algorithms (intuition only):

- Bully algorithm – highest ID wins
- Ring algorithm – token circulates

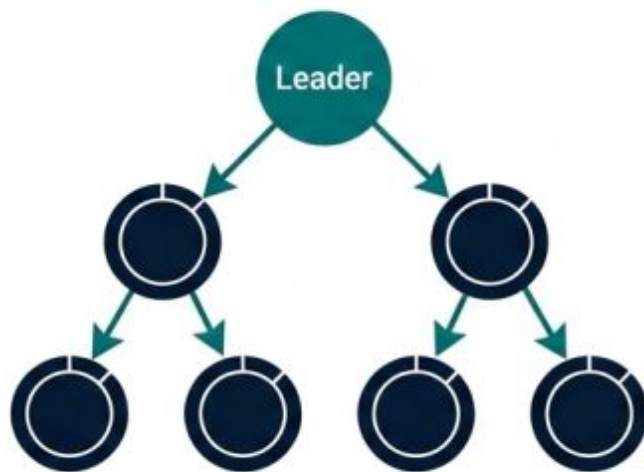
Modern Reality: Most systems now use Raft or ZooKeeper for leader election (we'll explore both in later weeks).

Establishing Authority

State 1: Chaotic Cluster



State 2: Structured Hierarchy



A single **coordinator** must resolve conflicting operations (e.g., replication, task assignment).

- **Classic Intuition:** **Bully** algorithm (highest ID wins) and **Ring** algorithms.
- **Modern Reality:** Production systems bypass these for dedicated **consensus protocols** (**Raft**) or robust coordination services (**ZooKeeper**).

Global Snapshots – Why It's Hard

“Photographing a moving system”

You want a consistent view of the entire distributed system at one logical moment.

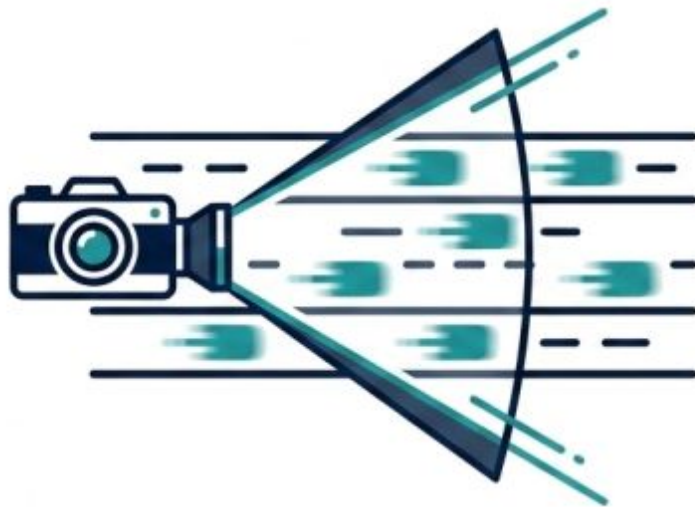
Problem: Messages are still in flight. Taking local snapshots at different wall-clock times gives inconsistent results (money can appear to disappear during a transfer).

This is exactly why distributed snapshot algorithms like Chandy-Lamport were invented.

Photographing a Moving System

How do we capture a consistent global state without freezing the entire system?

- **Local snapshots** at different wall-clock times yield **corrupt** global data.
- **In-transit messages** create **logical paradoxes** (e.g., money disappearing during transfers).
- **The Solution:** We must define a **logical "cut"** across all participating processes.



Chandy-Lamport Snapshot Algorithm

High-level intuition (no heavy proof)

1. A process decides to take a snapshot and sends marker messages on all outgoing channels.
2. When a process receives its first marker on a channel, it records its local state and starts recording messages on other channels.
3. When it has received markers on all channels, the snapshot is complete.

Result: A consistent cut of the global state (including in-transit messages).

The Chandy-Lamport Solution



1. Initiate: Process records local state, sends markers on all outgoing channels.

2. Receive First Marker: Process records local state, begins logging incoming messages.

3. Complete: Process has received markers on all incoming channels.

Captures a perfectly consistent global state, including all in-transit messages.

Coordination Services – ZooKeeper

From research to production

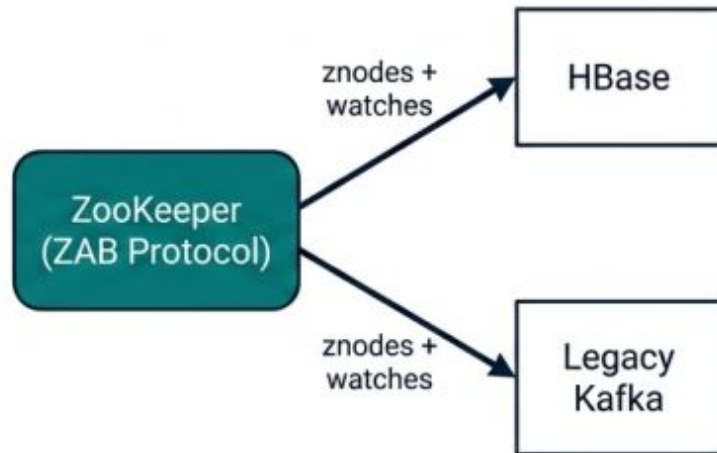
- Original goal: reliable coordination for large distributed systems (Yahoo)
- ZAB (ZooKeeper Atomic Broadcast) protocol — similar spirit to Raft
- znodes + watches = simple but powerful primitives
- Real usage: Kafka (historically), HBase, many Hadoop ecosystem tools

Modern Note: Kafka has largely moved from ZooKeeper to its own KRaft (Kafka Raft) implementation.

ZooKeeper & Beyond

Transitioning from academic research to global production systems.

- ZooKeeper: Reliable coordination utilizing the ZAB (Atomic Broadcast) protocol.
- Primitives: Exposes znodes + watches for simple, robust state management.
- Evolution: Modern systems are migrating from external ZooKeeper to internal Raft implementations (e.g., Kafka's KRaft).



Lab Preview – Build It Yourself

Two problem sets (due over the next 10 days)

Problem Set A – Vector Clocks (Core)

- Implement VectorClock class in Python
- Simulate message exchange between 3–5 processes
- Implement compare() that returns before / after / concurrent
- Test suite provided

Problem Set B – Clock Synchronization Comparison (Optional)

- Simulate Lamport vs NTP-style offset calculation
- Measure how often they disagree on ordering

The vector clock implementation directly supports later replication and consensus labs.

Summary & Key Takeaways

1 Physical time is useful but insufficient for ordering events across machines.

2 Lamport clocks capture causality with a single counter using the happens-before relation.

3 Vector clocks add precision by tracking knowledge per process and can detect true concurrency.

4 Coordination primitives (mutex, leader election, snapshots) build on logical time concepts.

5 Real systems combine these ideas (ZooKeeper, Raft, Spanner, Kafka) — you don't have to build everything from scratch.

References & Further Reading

1. Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. CACM.
2. Chandy, K.M. & Lamport, L. (1985). Distributed snapshots. ACM TOCS.
3. Mattern, F. (1989). Virtual time and global states of distributed systems.
4. Ongaro, D. & Ousterhout, J. (2014). In search of an understandable consensus algorithm (Raft).
5. Hunt, P. et al. (2010). ZooKeeper: Wait-free coordination for Internet-scale systems. USENIX ATC.
6. Corbett, J.C. et al. (2012). Spanner: Google's globally-distributed database. OSDI.
7. Tanenbaum & van Steen (4th ed.), Chapter 5 – Coordination.
8. Coulouris et al. (5th ed.), Chapter 14 – Time and Global States.
9. NTP RFC 5905.
10. raft.github.io (interactive visualizations – highly recommended)

Thank You

Questions & Discussion

Discussion prompt: Which concept from these two weeks (Lamport clocks, vector clocks, or snapshots) do you think will be most useful in your future projects or career, and why?

[Instructor Name]
[Email] · Office Hours: [time]